DSU W22 MODEL ANSWER MSBTE

This Study Material is provided by Campusifyplus.in ©

Made by Team Campusify

THIS IS THE FREE STUDY MATERIAL BY CAMPUSIFY!!

Be Part of Our Community

Telegram Channel - https://t.me/Campusifyy

Whatsapp Channel - https://tinyurl.com/Campusify-channel

Campusify is building the Biggest Diploma/Engineering Student Community in Maharashtra. Be part of the movement!

Join Channel for MSBTE Study Materials



Earn by Referring Campusify Course

Presented by **Campusify** | **10K** Active Members | **2.7L** Students Helped within **1 year**

Refer Our course with your **Third year** friend or cousin and get **10%** commission on each referral.

For More Details : WhatsApp Us at **+91 8291934640**

TY Course Link - https://www.campusifyplus.site

Join Channel for MSBTE Study Materials

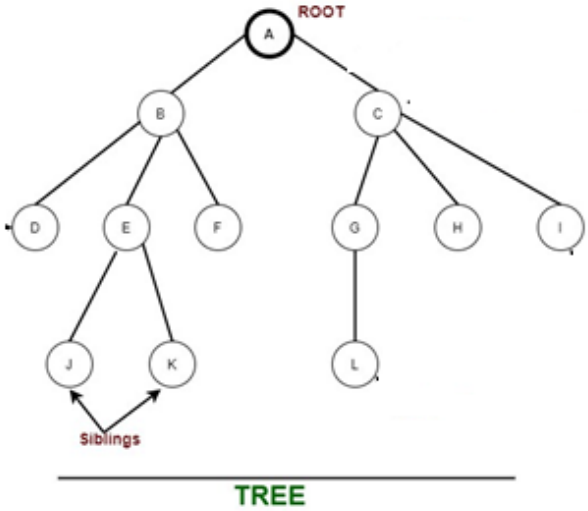
WINTER – 2022 EXAMINATION

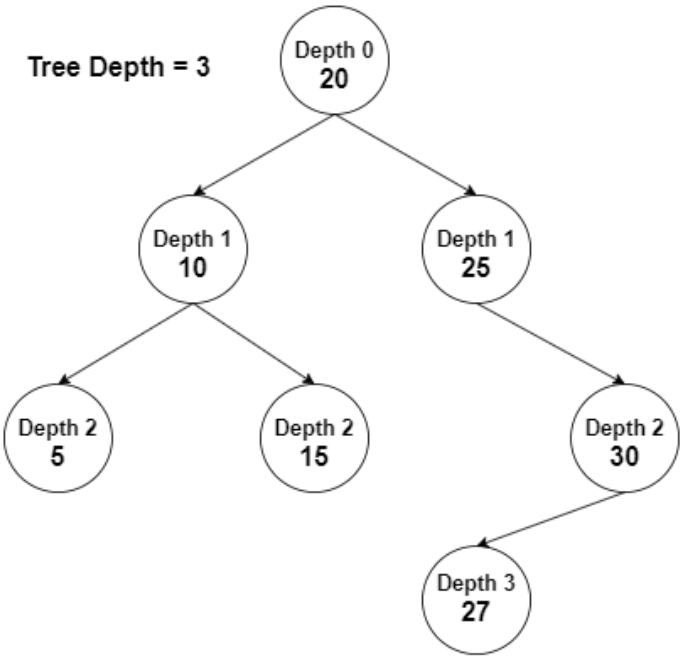
Subject Name: Data Structures Using 'C' **Model Answer** **Subject Code:** 22317

Important Instructions to examiners:

1) The answers should be examined by key words and not as word-to-word as given in the model answer scheme.
2) The model answer and the answer written by candidate may vary but the examiner may try to assess the understanding level of the candidate.
3) The language errors such as grammatical, spelling errors should not be given more Importance (Not applicable for subject English and Communication Skills.
4) While assessing figures, examiner may give credit for principal components indicated in the figure. The figures drawn by candidate and model answer may vary. The examiner may give credit for any equivalent figure drawn.
5) Credits may be given step wise for numerical problems. In some cases, the assumed constant values may vary and there may be some difference in the candidate's answers and model answer.
6) In case of some questions credit may be given by judgement on part of examiner of relevant answer based on candidate's understanding.
7) For programming language papers, credit may be given to any other program based on equivalent concept.
8) As per the policy decision of Maharashtra State Government, teaching in English/Marathi and Bilingual (English + Marathi) medium is introduced at first year of AICTE diploma Programme from academic year 2021-2022. Hence if the students in first year (first and second semesters) write answers in Marathi or bilingual language (English +Marathi), the Examiner shall consider the same and assess the answer based on matching of concepts with model answer.

Q. No.	Sub Q. N.	Answer	Marking Scheme
1		**Attempt any FIVE of the following:**	**10 M**
	a)	**Define complexity and classify it.**	**2 M**
	Ans	Complexity of an algorithm is a measure of the amount of time and/or space required by an algorithm for an input of a given size (n). Types of Complexity 1)Time complexity 2)Space complexity	Complexity definition-1 M and type - 1 M
	b)	**State the following terms :** **(i) searching** **(ii) sorting**	**2 M**
	Ans	(i) Searching: The process of finding the desired information from the set of items stored in the form of elements in the computer memory is referred to as 'searching in data structure' (ii) Sorting: Sorting is the processing of arranging the data in ascending and descending order.	Each term carries – 1 M

	c)	**List any four applications of stack.**	**2 M**
	Ans	1)Reversing a list 2)Conversion of infix to prefix expression 3)Conversion of infix to postfix expression 4)Evaluation of prefix expression 5)Evaluation of postfix expression 6)Tower of Hanoi 7)Polish notation	Four applications carries - 2 M
	d)	**List any four types of queue.**	**2 M**
	Ans	1)linear queue 2)circular queue 3)Priority queue 4)Dequeue or double ended queue	Each type ½ M
	e)	**Define Abstract data type.**	**2 M**
	Ans	• ADT is defined as a mathematical model of the data objects that make up a data type as well as functions that operate on these objects. • ADT is the specification of logical and mathematical properties of a data type or structure.	Correct definition 2 M
	f)	**Define the following terms :** **(i) Sibling** **(ii) Depth of tree**	**2 M**
	Ans	**i) Sibling:** **Siblings:** Nodes which belong to the same parent are called as siblings. In other words, nodes with the same parent are sibling nodes.  TREE	Each term carries 1 M with example

		(ii) Depth of tree The **depth** of a node is the number of edges from that node to the tree's root node. As such, the depth of the whole tree would be the depth of its deepest leaf node. The root node has a depth of **0**. 	
	g)	**Write algorithm for preorder traversal of binary tree.**	**2 M**
	Ans	Step 1 : Start Step 2: Repeat Steps 3 to 5 while TREE != NULL Step 3: Write TREE -> DATA Step 4: PREORDER(TREE -> LEFT) Step 5: PREORDER(TREE -> RIGHT) [END OF LOOP] Step 6: END **Preorder:** The preorder traversal method performs the following operations: Process the root node (N). Traverse the left subtree of N (L). (c) Traverse the right subtree of N (R).	Correct algorithm – 2 M
2.		**Attempt any <u>THREE</u> of the following:**	**12 M**
	a)	**Write a program to implement bubble sort.**	**4 M**
	Ans	/* *Bubble sort code* */ #include <stdio.h> int main()	Correct logic - 4 M

```
{
 int array[100], n, c, d, swap;
 printf("Enter number of elements\n");
 scanf("%d", &n);
 printf("Enter %d integers\n", n);
 for (c = 0; c < n; c++)
  scanf("%d", &array[c]);
 for (c = 0 ; c < n - 1; c++)
 {
  for (d = 0 ; d < n - c - 1; d++)
  {
   if (array[d] > array[d+1]) /* For decreasing order use '<' instead of '>' */
   {
    swap       = array[d];
    array[d]   = array[d+1];
    array[d+1] = swap;
   }
  }
 }
 printf("Sorted list in ascending order:\n");
 for (c = 0; c < n; c++)
   printf("%d\n", array[c]);
 return 0;
}
```

	b)	**Convert following expression into postfix form with illustration of all steps using stack: (A +B-C+D*E/F^G)**	**4 M**

Ans

Character scanned	Stack	Postfix expression
(	(	Empty
(	((	Empty
A	((	A
+	((+	A
B	((+	AB
-	((-	AB+
C	((-	AB+C
+	((+	AB+C-
D	((+	AB+C-D
*	((+*	AB+C-D
E	((+*	AB+C-DE
/	((+	AB+C-DE*
F	((+/	AB+C-DE*F
^	((+/^	AB+C-DE*F
G	((+/^	AB+C-DE*FG
)	(	AB+C-DE*FG^/+
)	Empty	AB+C-DE*FG^/+

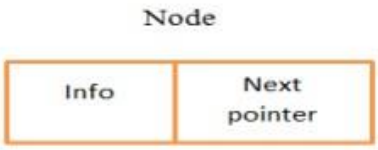
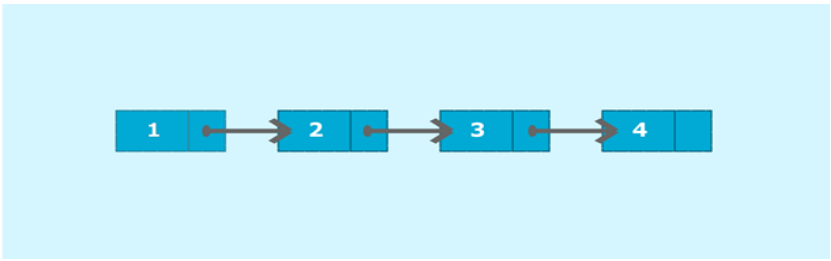
Stepwise carries 4 M

c) Differentiate between Stack and Queue (any four points). **4 M**

Ans

Stacks	**Queues**
Stacks are based on the LIFO principle, i.e., the element inserted at the last, is the first element to come out of the list.	Queues are based on the FIFO principle, i.e., the element inserted at the first, is the first element to come out of the list.
Insertion and deletion in stacks takes place only from one end of the list called the top.	Insertion and deletion in queues takes place from the opposite ends of the list. The insertion takes place at the rear of the list and the deletion takes place from the front of the list.
Insert operation is called push operation.	Insert operation is called enqueue operation.
Delete operation is called pop operation.	Delete operation is called dequeue operation.

Four points carries 4 M

		access the list, called the top, which always points to the last element present in the list.	access the list. The front pointer always points to the first element inserted in the list and is still present, and the rear pointer always points to the last inserted element.
		Stack is used in solving problems works on recursion.	Queue is used in solving problems having sequential processing.
		Stack does not have any types.	Queue is of three types – 1. Circular Queue 2. Priority queue 3. double-ended queue.
	d)	**Explain node structure for single linked list. Also write advantages of singly list over array. (any Two)**	**4 M**
	Ans	• **Node:** Each data element in a linked list is represented as a node. Node contains two parts one is info (data) and other is next pointer (address). Info part stores data and next pointer stores address of next node.  Singly Linked List The singly linked list is a linear data structure in which each element of the list contains a pointer which points to the next element in the list. Each element in the singly linked list is called a node. Each node has two components: data and a pointer next which points to the next node in the list. The first node of the list is called as head, and the last node of the list is called a tail. The last node of the list contains a pointer to the null. Each node in the list can be accessed linearly by traversing through the list from head to tail.  Consider the above example; node 1 is the head of the list and node 4 is the tail of the list. Each node is connected in such a way that node 1 is pointing to node 2 which in turn	Advantages 2 M and structure of singly linked list – 2 M

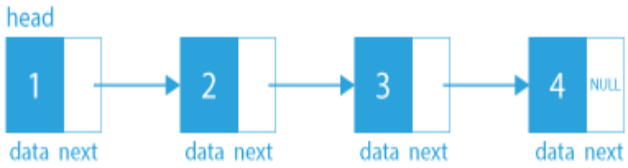
pointing to node 3. Node 3 is again pointing to node 4. Node 4 is pointing to null as it is the last node of the list.

Advantages of a Linked List over Array

Arrays allow random access and require less memory per element (do not need space for pointers) while lacking efficiency for insertion/deletion operations and memory allocation. On the contrary, **linked lists are dynamic and have faster insertion/deletion time complexities**

1) **Dynamic Data Structure:**

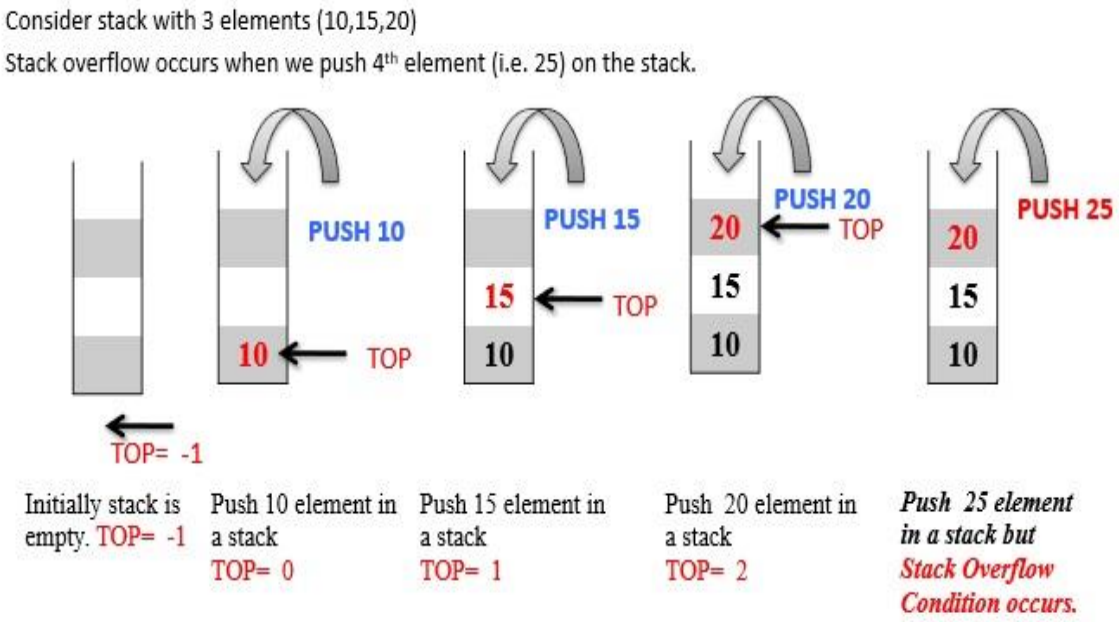
Linked List being a dynamic data structure can shrink and grow at the runtime by deallocating or allocating memory, so there is no need for an initial size in linked list.



Whereas an initial size has to be declared in an array, and the number of elements cannot exceed that size.

2) **No Memory Wastage:**
As the size of a linked list can grow or shrink at runtime, so there is no memory wastage. Only the required memory is allocated.
In arrays, we have to first initialize it with a size which we may or may not fully use; hence wastage of memory may occur.
3) **Implementation:**
Some very helpful data structures like queues and stacks can be easily implemented using a Linked List.
4) **Insertion and Deletion Operation:**
In a Linked List, insertion and deletion operations are quite easy, as there is no need to shift every element after insertion or deletion. Only the address present in the pointers needs to be updated.
While in an array, we have to shift elements. Suppose we have an array that is sorted, and now we need to insert some element in the array in a sorted way.
Let arr[]= [1, 3 , 5, 7, …..], and we have to insert 2. So, all the elements after 1 have to move by one place towards the right.

3.		**Attempt any <u>THREE</u> of the following:**	**12 M**
	a)	**Explain stack overflow and stack underflow with example.**	**4 M**

	Ans	**Stack Overflow:** Stack overflow caused by an insertion in a stack that is already full. Before you push an element into the stack, first check whether stack is full or not. If stack is full then stack overflow, else push element onto the stack. if (isfull()= =1) then stack overflow , else push( ) element on the stack. **Example:**  **Fig. Stack Overflow** **Stack Underflow:** Stack underflow caused by deletion of an element from an empty stack. if (isempty()= =1) then stack Underflow , else pop( ) element from the stack. **Example:**	Stack overflow with diagram-2M Stack underflow with diagram-2M

Consider stack with 3 elements (10,15,20)

Stack underflow occurs when we pop an element from the empty stack.

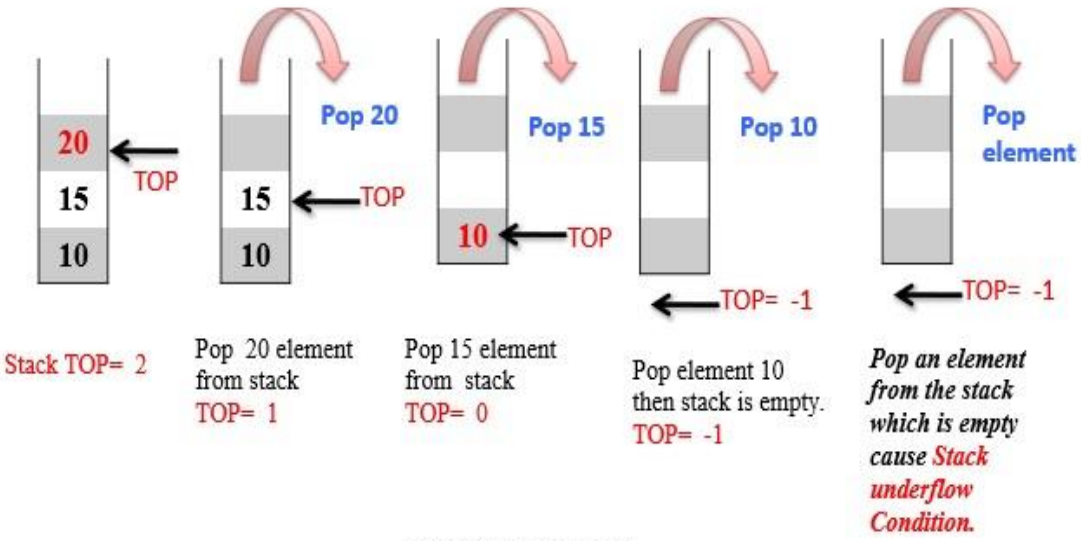
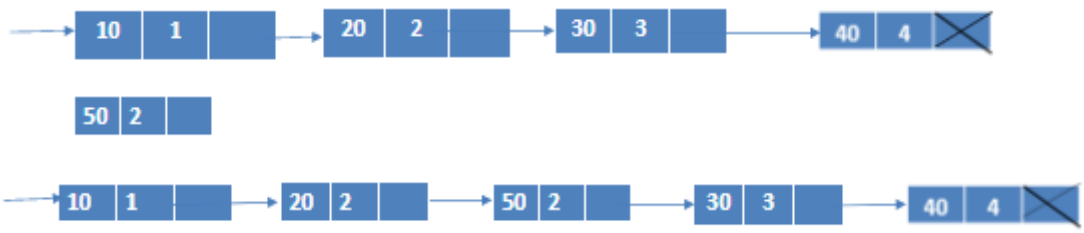


Fig. Stack Underflow

	b)	**With a neat sketch explain working of priority queue.**	**4 M**
	Ans	**Priority Queue:** • A priority queue is a data structure in which each element is assigned a priority. The priority of the element will be used to determine the order in which the elements will be processed. • The general rules of processing the elements of a priority queue are • An element with higher priority is processed before an element with a lower priority. • Two elements with the same priority are processed on a first-come-first-served (FCFS) basis. • A priority queue can be thought of as a modified queue in which when an element has to be removed from the queue, the one with the highest-priority is retrieved first. 	Diagram 2M Explanation 2M

Operation on priority queue

1) Insertion in priority queue
- When a new element has to be inserted in a priority queue, we have to traverse the entire list until we find a node that has a priority lower than that of the new element.
- The new node is inserted before the node with the lower priority. However, if there exists an element that has the same priority as the new element, the new element is inserted after that element.

2) Deletion in priority queue
- It is a very simple process in this case.
- The first node of the list will be deleted, and the data of that node will be processed first.

c) Find location of element 20 by using binary search algorithm in the list given below: **4 M**

10, 20, 30, 40, 50, 60, 70, 80

Ans **Binary Search:** Each correct step-1M

Array A: {10,20,30,40,50,60,70,80} and key element 20

Step 1: Store given sorted elements in an array say A[8].

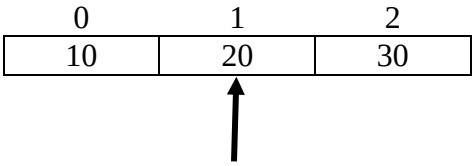
0	1	2	3	4	5	6	7
10	20	30	40	50	60	70	80

Step 2:
- First set Begin and End.
- Begin(i)=0
- End(j)=7
- Mid=(Begin + End)/2=(0+7)/2=3.5=3
- Compare A[Mid]= =key
- Here A[Mid]=A[3]=40 & key=20
- Key< A[Mid] i. e 20<40

0	1	2	3	4	5	6	7
10	20	30	40	50	60	70	80

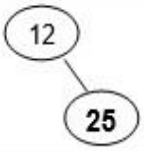
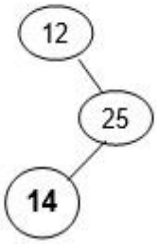
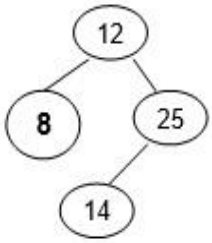
Mid

Step 3:
- key can be found in left side of Mid element.
- Beginning remain same and end will change to Mid-1.

		• Begin(i)=0 • End(j)=Mid-1=3-1=2 • Mid=(Begin + End)/2=(0+2) / 2=2 / 2=1  **Mid** Step 4: • Compare A[Mid]= =key • Here A[Mid]=A[1]=20 & key=20 • Key == A[Mid] i. e 20 = =20 **Thus element is found at A[1] location in an array [10,20,30,40,50,60,70,80]**	
	d)	**Explain Binary Search Tree (BST) with example.**	**4 M**
	Ans	**Binary Search Tree (BST):** **A binary tree which is either empty or in which each node contains a key that satisfies the following condition-** 1. All keys are distinct. 2. For every node 'x' in the tree the value of all the keys in its left sub tree are smaller than the key value in 'X' 3. For every node 'x' in the tree the value of all the keys in its right sub tree are larger than the key value in 'X'. In BST, Nodes are arranged such that all nodes in a left sub tree having less values than the root node & all the nodes in right subtree having values greater than root node. **Characteristics of BST** 1. All the nodes in a left sub tree having less values than root node. 2. All the nodes in a right sub tree having greater values than root node. so faster insertion, deletion & searching can be performed.	Explanation= 2M Example=2M

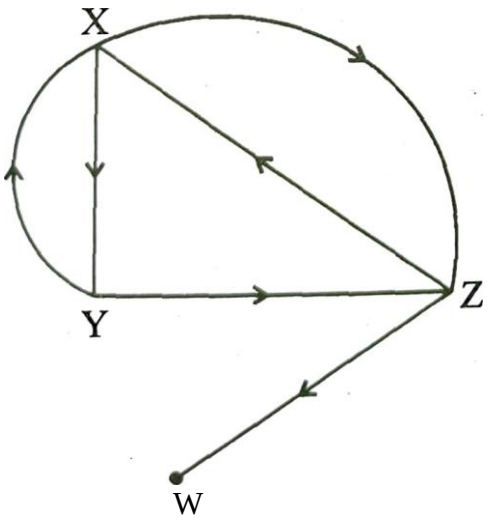
Example:

BST for following Data: 12, 25, 14, 8

Sr. No	Input Element	Operation/Description	Tree
1.	**12**	First element Insert into tree	
2.	**25**	Compare 25 with 12 i.e., 25>12 so goes to right of 12	
3.	**14**	Compare 14 with 12 i.e., 14>12 so goes to right of 12 But there is 25 so compare 25 with 14 i.e.,14<25 so, 14 goes to left of 25	
4.	**8**	Compare 8with 12 i.e., 8<12 so goes to left of 12	

4.		**Attempt any <u>THREE</u> of the following:**	**12 M**
	a)	**Differentiate between linear and non-linear data structure. (any four points)**	**4 M**
	Ans		Any 4 valid points: 1 M each

Sr.	Linear data structure	Non- Linear data structure
1	Data elements are arranged in a linear order	Data elements are arranged in a non-linear (hierarchical) order
2	Single level is involved.	Multiple level is involved.
3	Implementation is easy	Implementation is complex
4	Memory is not utilized in an efficient way.	Memory is utilized in an efficient way.
5	Examples are: array, stack, queue, linked list, etc.	Examples are: trees and graphs.
6	Application: software development.	Application: Artificial Intelligence and image processing.

	b)	**Consider the graph given below:**  **(i) Find indegree(x)** **(ii) Find outdegree(z)** **(iii) Find sink node** **(iv) Successor of node y**	**4 M**
	Ans	Indegree (x): 2 Outdegree (z): 2 Sink Node: W Successor of node y: Z	1M each correct answer
	c)	**Describe working of linear search with example.**	**4 M**

Ans **Working of Linear Search:**

In linear search, elements are examined sequentially starting from first element.
It traverse data sequentially to find particular data.
The process of searching end when comparison results in success.
Searching continues sequentially till the record with matching key(i.e. element to be search)is compared with other elements.
Compare key with each element one by one .
Not Suitable for searching when large number of elements are given.
Time consuming.

Working-2M

Example-2M

Example:

Consider the array A={23,16,5,105}
Find key=5 is present or not.

Index	Element
0	23
1	16
2	5
3	105

Key=5
Compare
A[0]==5
No match

Index	Element
0	23
1	16
2	5
3	105

Key=5
Compare
A[1]==5
No match

Index	Element
0	23
1	16
2	5
3	105

Key=5
Compare
A[2]==5
Match Found

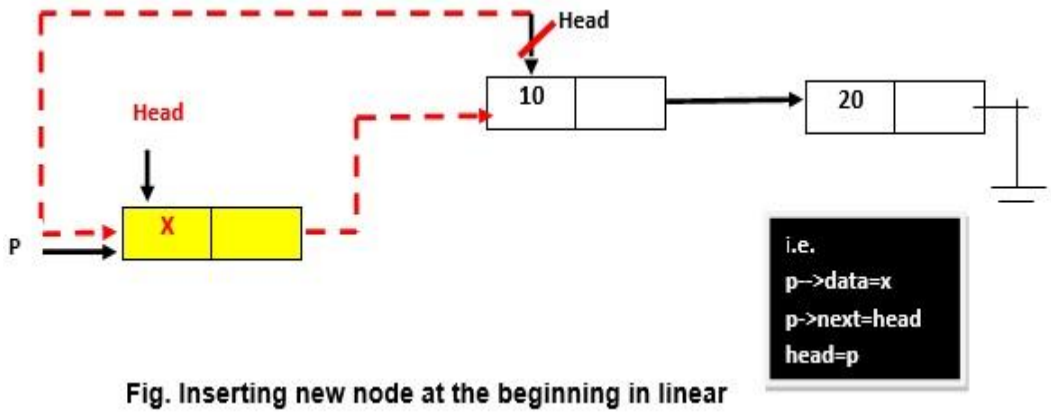
Thus, key element 5 is found at A[2] i.e. in the third location

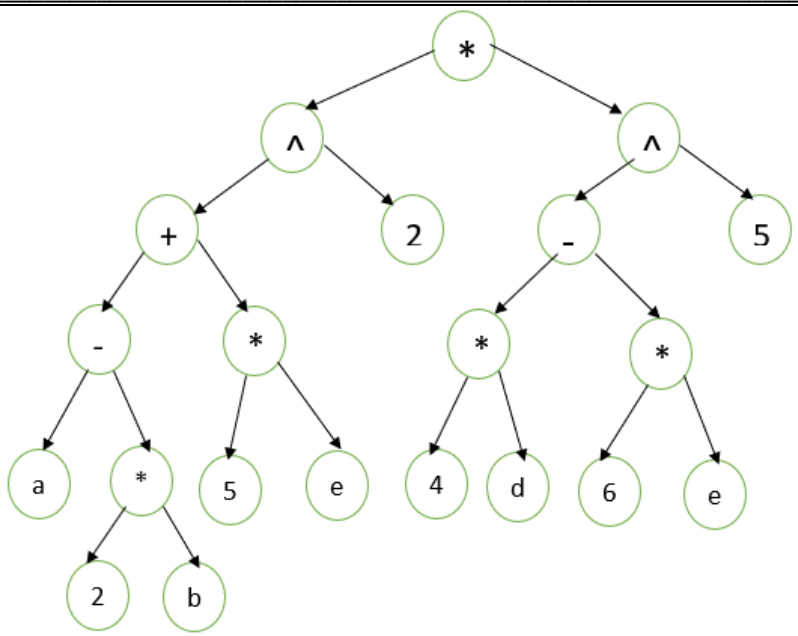
d) **Compare linear list with circular list.** **4 M**

Ans

Sr.	Linear List	Circular List
1	Next field of the last node is set to NULL	Next field of the last node is connected back to head node.
2	Points to first node	Points to last node.
3	Insertion at end has O(n)	Insertion at end has O(1) complexity

Any 4 points: 1 M each

				complexity.	
			4	Ends on Finding the NULL pointer	Searching terminates on coming to first node.
			5	Not suited for implementing data structure.	Can be used for Queue data structure.
	e)	**Write an algorithm to insert a new node at the beginning in linear list.**			**4 M**
	Ans	**Algorithm to insert a new node at the beginning in linear list:** 1. START 2.Obtain free space for new node 3. Assign data to the data field of new node. 4.Set the next field of new node to the beginning of the list. 5.Change the reference pointer(head) of the linked list to point new node. 6.STOP  **Fig. Inserting new node at the beginning in linear**			4 M for insert a new node at the beginning in linear list
5.		**Attempt any <u>TWO</u> of the following:**			**12 M**
	a)	**Draw tree for given expression :** **$(a-2b+5c)^2 * (4d -6e)^5$.**			**6 M**

	Ans		Correct tree structure - 6 M
	b)	**Write a 'C' program for insert and delete operation to be performed on queue.**	**6 M**

	Ans	**Program:**
		//Implementation of queue as Data Structure using array;
		#include <stdio.h>
		#define qsize 5
		int front;
		int rear;
		int queue[qsize];
		void insertq(int);// function declaration
		void deleteq();
		void display();
		int main()
		{
		//initially queue is empty
		front = rear =-1;
		int x,choice;
		do
		{
		printf("Queue Operations\n");
		printf("1.Insert\n2.Delete\n3.Display\n4.Exit");
		printf("Enter your choice 1\2\3\4");
		scanf("%d",&choice);
		switch(choice)
		{
		case 1: {
			6M for correct program

```
printf("Enter value to be added\n");
scanf("%d",&x);
insertq(x);
break;
}
case 2: {
deleteq();
break;
}
case 3: {
display();
break;
}
case 4: {
printf("Exit\n");
break;
}
default: printf("Enter Valid choice from 1/2/3/4\n");
break;
 }
}while(choice !=4);
return 0;
}
void insertq(int x)
{
//check queue is full?yes or no
if(rear == qsize-1)
printf("Queue is Overflow\n");
else
{
//queue with zero element
if(front ==-1 && rear==-1)
{
front=0;
rear =0;
}
else
{
// queue with one or more element
rear = rear +1;
}
 // add new element at rear end of queue
queue[rear]=x;
```

```
 }
}
void deleteq()
{
 int x;
 //check queue underflow? yes or no
 if (front==-1 && rear==-1)
 {
 printf("queue is underflow\n");
  }
 else
 {
 // identify element to be processed
 x= queue[front];
 //is it the last element
 if(front==rear)
 {
 front=rear=-1;// queue is empty
 }
 else
 {
 front = front+1;
 }
  printf("element %d is deleted\n",x);
 }
}
void display()
{
 int i;
 if(front==-1 && rear ==-1)
 {
 printf("Queue is Empty\n");
 }
 else
 {
 printf("Queue elements are\n");
 for(i=front; i<=rear; i++)
 {
 printf("%d\t",queue[i]);
 }
 }
}
```

	c)	**Write a 'C' program for insertion sort.** **Sort the following array using insertion sort: 30 10 40 50 20 45**	**6 M**

	Ans	**Program to implement insertion sort:**
		#include <stdio.h>
		int main()
		{
		int a[10],n,i,min,j,key;
		printf("Enter number of elements\n");
		scanf("%d",&n);
		printf("enter %d elements\n",n);
		for (i=0;i<n;i++)
		{
		scanf("%d",&a[i]);
		}
		for(i=1;i<=n-1;i++)
		{
		key= a[i];
		for (j=i-1 ; j >=0 && a[j] > key ;j--)
		{
		a[j+1]= a[j];
		}
		a[j+1]= key ;
		}
		printf("Sorted Array\n ");
		for (i=0;i<n;i++)
		{
		printf("%d\t",a[i]);
		}
		return 0;
		}
		Example:
			Insertion Program-3 M
			Example with all iterations -3 M

pass-1	**30**	10	40	50	20	45
	SA	**USA**				

10	**30**	40	50	20	45
SA		**USA**			

pass-2	**10**	**30**	40	50	20	45
	SA		**USA**			

10	**30**	**40**	50	20	45
SA			USA		

pass-3

10	**30**	**40**	50	20	45
SA			**USA**		
10	**30**	**40**	**50**	20	45
SA				**USA**	

pass-4

10	**30**	**40**	**50**	20	45
SA				**USA**	
10	30	40	50	**50**	45
SA				**USA**	
10	30	40	**40**	**50**	45
SA				**USA**	
10	30	**30**	**40**	**50**	45
SA				**USA**	
10	**20**	**30**	**40**	**50**	45
SA					**USA**

pass-5

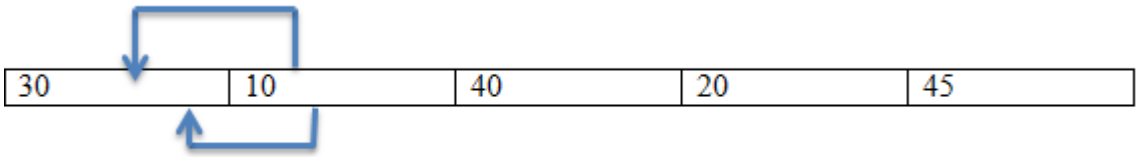
10	**20**	**30**	**40**	**50**	45
SA					**USA**
10	**20**	**30**	**40**	**50**	**50**
SA				**USA**	
10	**20**	**30**	**40**	**45**	**50**
SA					

OUTPUT:

10	**20**	**30**	**40**	**45**	**50**
SA					

OR

Pass1: 1st element is compared with all previous element (i.e. 0th element)



10	30	40	20	45

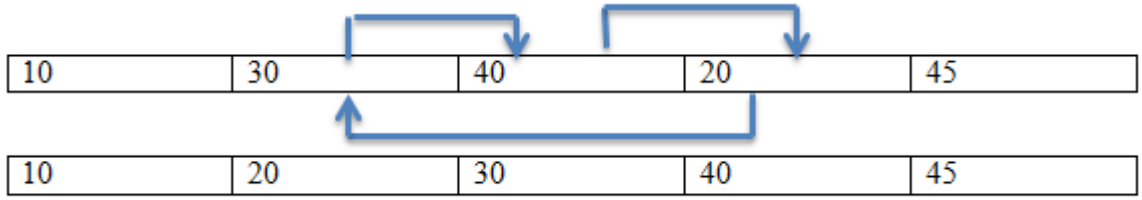
Since 30>10, move 30 to right and insert (place) 10 to its correct position.

Pass 2: Second element is compared with all previous element (i.e. 0th and 1st)



Since 10<40 and 30<40, 40 is at its correct position.

Pass 3: 3rd element is compared with all previous elements (i.e. 0th, 1st and 2nd)



Since 40>20, move 40 to right and compare 30 with 20 again. Again 30>20 move 30 to right and compare 10 with 20. As 10<20 insert (place) 20 to a[1] position.

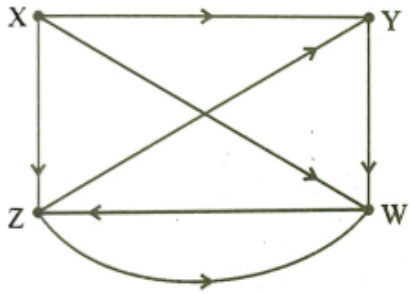
Pass 4: 4th element is compared with all its previous elements(i.e. 0th, 1st , 2nd and 3rd)

10	20	30	40	45

Since 10<45, 20<45, 30<45 and 40<45, 45 is at its correct position.

Sorted array is

10	20	30	40	45

6.		**Attempt any <u>TWO</u> of the following:**	**12 M**
	a)	**Consider the graph G given below:**  **i) Write Adjacency matrix representation.** **ii) Write Adjacency list.**	**6 M**
	Ans	**Adjacency matrix representation.** **W X Y Z** **W** [0 0 0 1] **X** [1 0 1 1] **Y** [1 0 0 0] **Z** [1 0 1 0] **Adjacency list.** **Header Node** | **Adjacent Node list** W | Z X | W, Y, Z Y | W Z | W, Y	Correct Adjacency Matrix-3 M Correct Adjacency List-3 M
	b)	**Write a menu driven 'C' program to implement stack using array with the following menu: (i)push (ii)pop (iii)display (iv)exit**	**6 M**

	Ans	**Menu driven program to implement stack operations**
		#include<stdio.h>
		#define stacksize 5
		int stack[stacksize];
		int top;
		void push(int x);
		void pop();
			Each operation with correct logic code -1 M each
			
			Main function and

```
void display();
int main()
{
   //clrscr();
   top=-1;
   int choice,x,i;
      printf("\n\t STACK OPERATIONS USING ARRAY");
   printf("\n\t--------------------------------");
   printf("\n\t 1.PUSH\n\t 2.POP\n\t 3.DISPLAY\n\t 4.EXIT");
   do
   {
      printf("\n Enter the Choice:");
      scanf("%d",&choice);
      switch(choice)
      {
         case 1:
         {
            printf("Enter value to be pushed into stack\n");
            scanf("%d",&x);
            push(x);
            break;
         }
         case 2:
         {
            pop();
            break;
         }
         case 3:
         {
            display();
            break;
         }
         case 4:
         {
            printf("\n\t EXIT POINT ");
            break;
         }
         default:
         {
            printf ("\n\t Please Enter a Valid Choice(1/2/3/4)");
         }
         }
   }
```

stack initialization- 2 M

```
   while(choice!=4);
   return 0;
}
void push(int x)
{
   if(top== stacksize-1)
   {
      printf("\n\tSTACK is over flow");
         }
   else
   {
       top++;
      stack[top]=x;
   }
}
void pop()
{
   int x;
   if(top==-1)
   {
      printf("\n\t Stack is under flow");
   }
   else
   {
      x= stack[top];
      printf("\n\t The popped elements is %d",x);
      top--;
   }
}
void display()
{
   if(top>=0)
   {
      printf("\n The elements in STACK \n");
      for(int i=top; i>=0; i--)
         printf("\n%d",stack[i]);
      printf("\n Press Next Choice");
   }
   else
   {
      printf("\n The STACK is empty");
   }
  }
```

	c)	**Write the 'C' function for:** **(i) searching a node in single linked list.** **(ii) counting number of nodes in single linked list.**	**6 M**

	Ans	**I. searching a node in single linked list.**
		void search( )
		{
		printf("enter data to be searched\n");
		scanf("%d", &data);
		
		struct node *ptr;
		int pos=1;
		ptr=start;
		while(ptr!=NULL)
		{
		if(ptr->info==x)
		{
		printf("element %d is found at position %d\n",x,pos);
		break;
		}
		else
		{
		ptr=ptr->next;
		pos++;
		}
		}
		if(ptr==NULL)
		printf("element %d not found in list\n",x);
		}
		
		II. Counting number of nodes in single linked list.
		void count()
		{
		struct node *ptr;
		int c=0;
		ptr=start;
		while(ptr!=NULL)
		{
		ptr=ptr->next;
		c++;
		}
		printf("no of nodes in list are %d\n",c); }
			Correct function for searching node – 3 M
			
			Correct function for Counting nodes – 3 M